

ENTERPRISE AI SUMMIT 2026

Adversarial Code Assurance as a Way Back to The Paved Road

...and finally trusting your AI code.

James Wickett

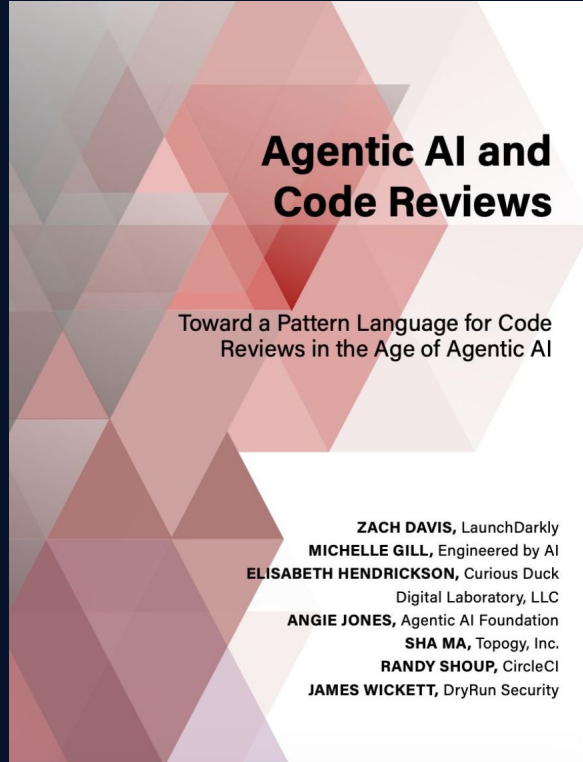
/in/wickett @wickett



**DRYRUN
SECURITY**



Agentic AI and Code Reviews



- Small Incremental Changes
- Shift Left
- Design for Verification
- Deterministic Engineering Practices
- Context Engineering
- Continuous Code Quality Loop
- Adversarial LLM Review
- Multi-Agent Reviews
- Spec-Driven Development

AI Software Factories have broken AppSec

87%

*of PRs produced by
Claude, Codex, & Gemini
contain vulns*

The Agentic Coding Security Report, Mar 2026

5x

*code volume in the
enterprise in the next
18-24 months*

78%

*of AI-made flaws are
undetectable by SAST*

Internal sample of 20,000 PRs, Jan 3-9 2026

The AI Software Factory Creates Proportional Demand for Generation & Verification

AI Code Generation is the Wild West right now

PRODUCTION

Unverified security quality. No testing for AI-specific vulnerabilities. No policy enforcement. No auditability. Limited to no observability.



Last gen appsec tools: overloaded, limited coverage & exponential noise

GENERATION LAYER

OpenAI Codex

Claude Code

Gemini Code

Cursor

GitHub Copilot

The Paved Road

Jason Chan · Netflix

2013: From Gates to Guardrails

2016: The Psychology of Security Automation

Build a security supported way to do the work safely.



James Wickett
/in/wickett · @wickett



Paved Road > Shift Left

Desired Feature: Alice belongs to Acme and wants to view an invoice

```
SELECT id, tenant_id, amount_cents, memo  
FROM invoices WHERE id = ?
```

Wait, Alice can read invoices from other tenants...



<https://www.youtube.com/watch?v=3TNpOD6bov8>

Make the isolation boundary testable

First repair: the caller can still choose the tenant.

```
tenant_id = environ.get(  
    "HTTP_X_TENANT_ID", principal.tenant_id)
```

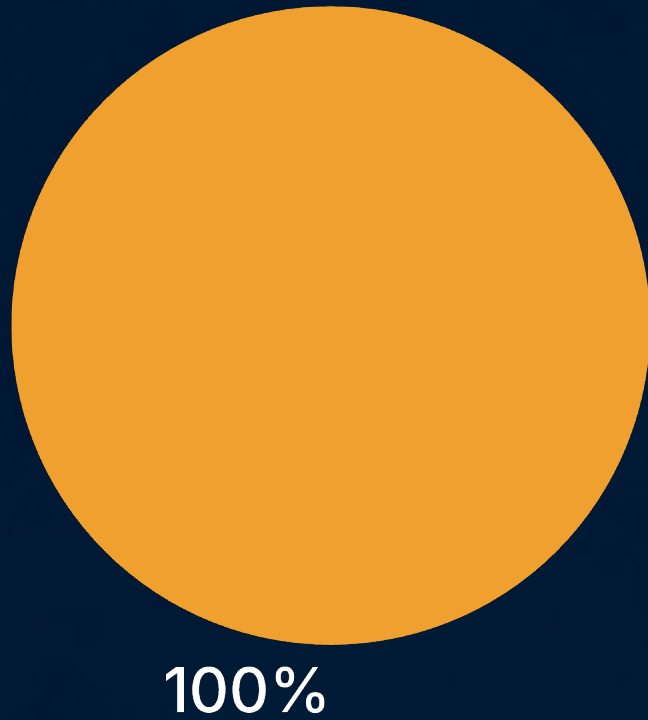
Verified repair: `principal.tenant_id`

Alice → Globex: denied · Alice → Acme: allowed

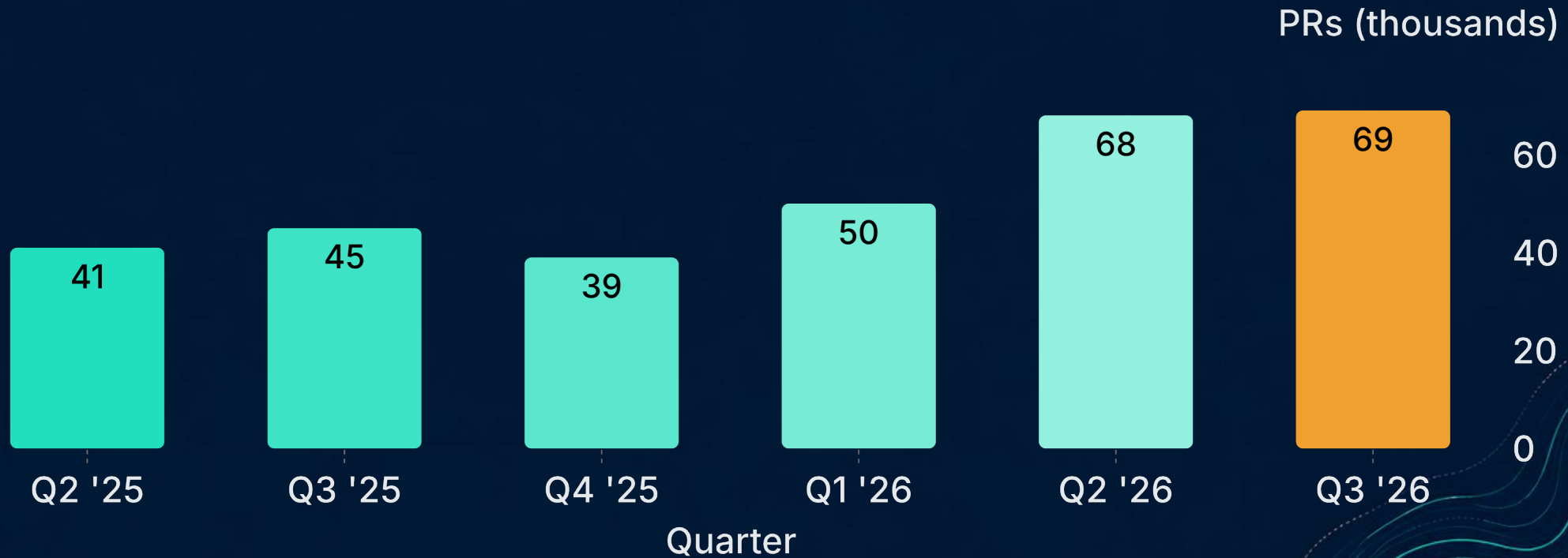
Re-test and review the revised commit.



AI Coding Adoption



Pull request volume grew 1.7x



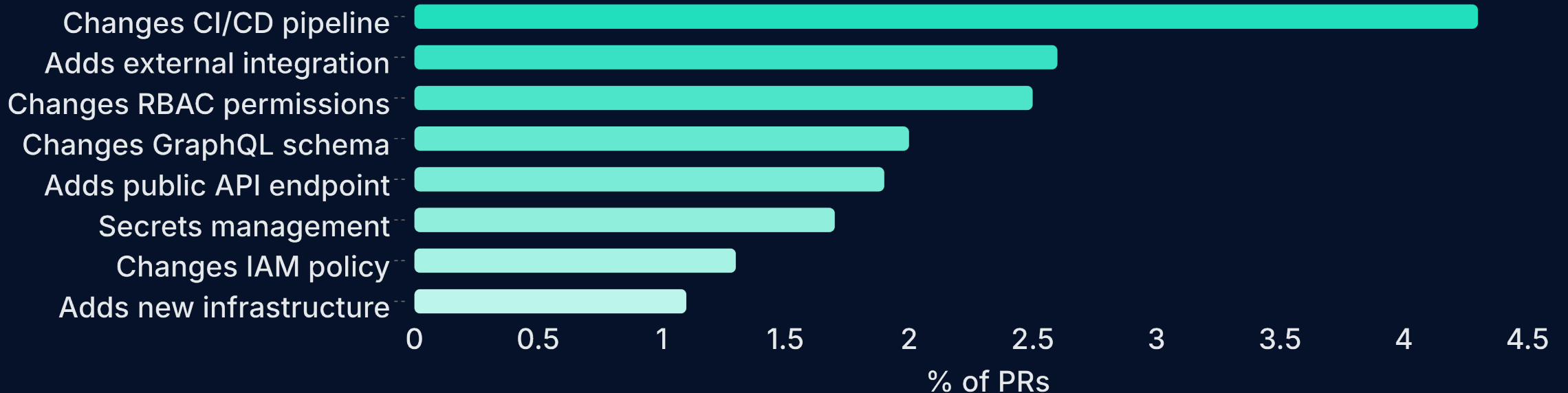
342 repositories, Q2 2025–Q3 2026

Organizations with Agentic Software Factory Activity



1 in 5 PRs changes security posture

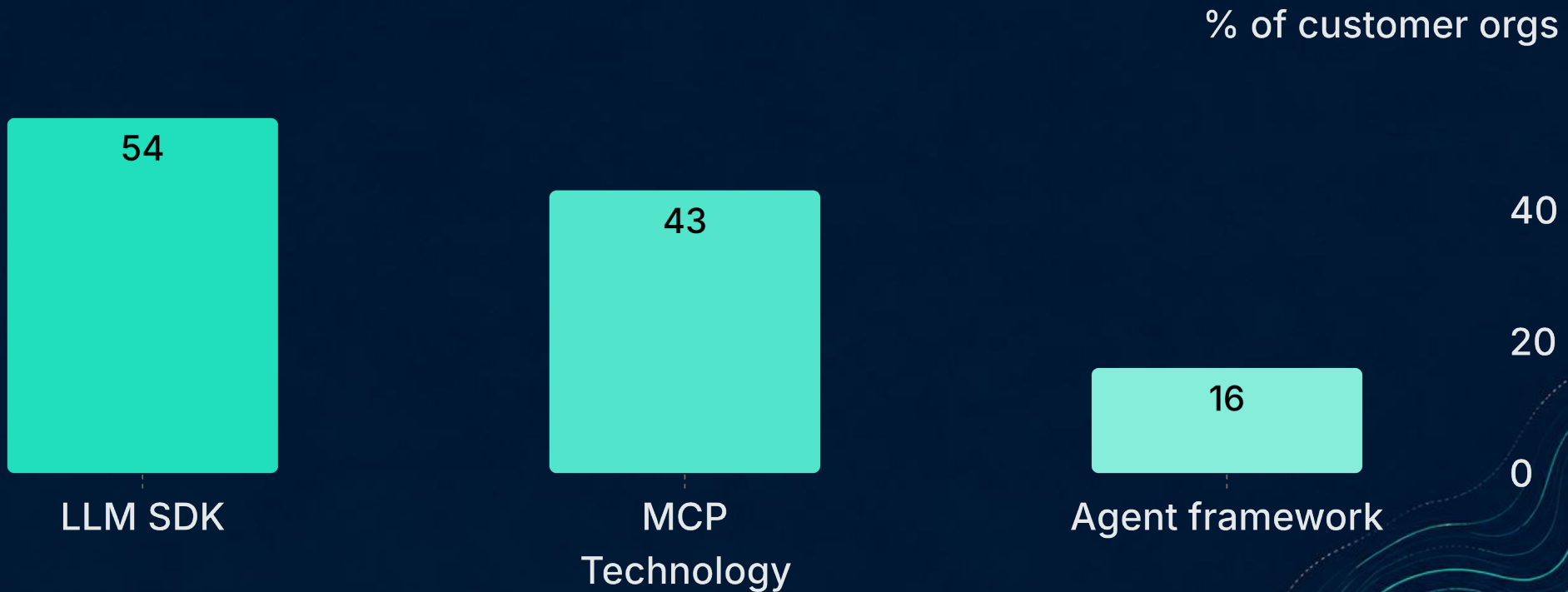
Category



Bonus: 1 in 13 adds attack surface

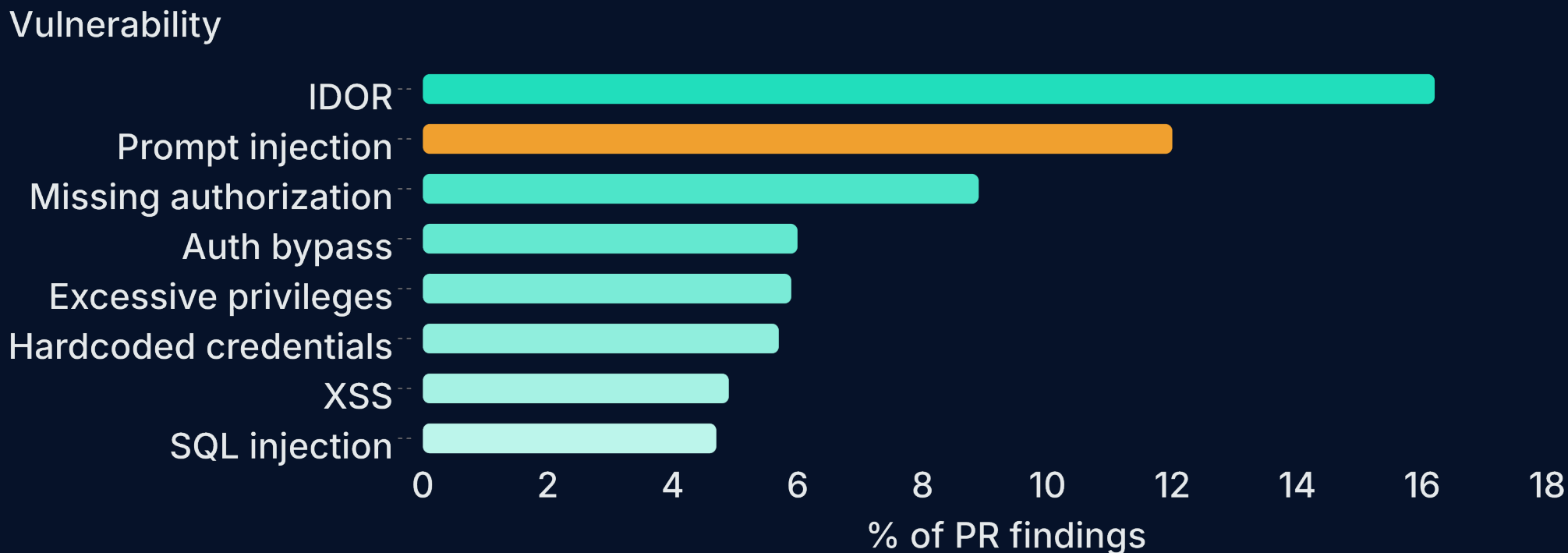
Source DryRun data analysis of 198k anonymized PRs

AI dependencies are already widespread



Anonymous data from 74 sampled organizations consisting of 1,906 randomized repositories.

AI-specific risks join familiar vulnerabilities



Adversarial review challenges the change

Understand the system

Code history, context, and security boundaries

Challenge independently

Find a counterexample to the author's assumptions

Exploit Chaining

Able to combine exploits and re-prioritize

Apply security contract

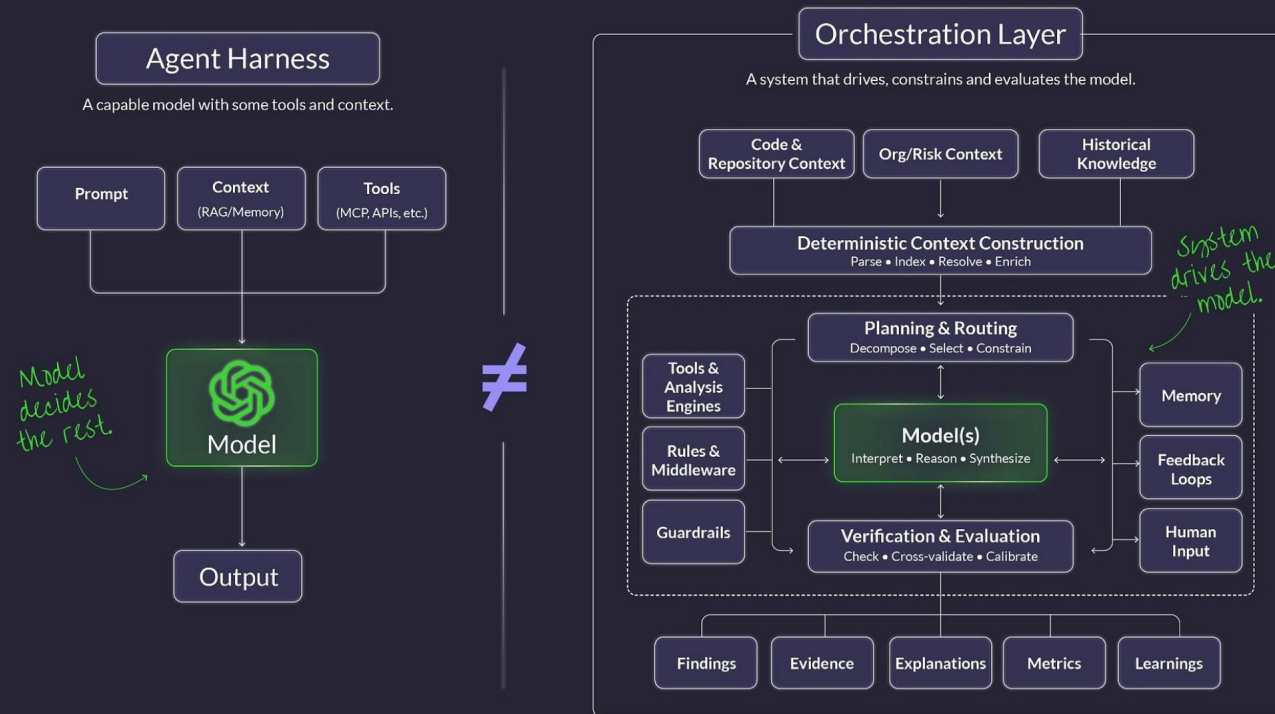
Baseline checks plus product-specific rules

Verify the repair

Re-test and review the revised commit

Harness ≠ Orchestration

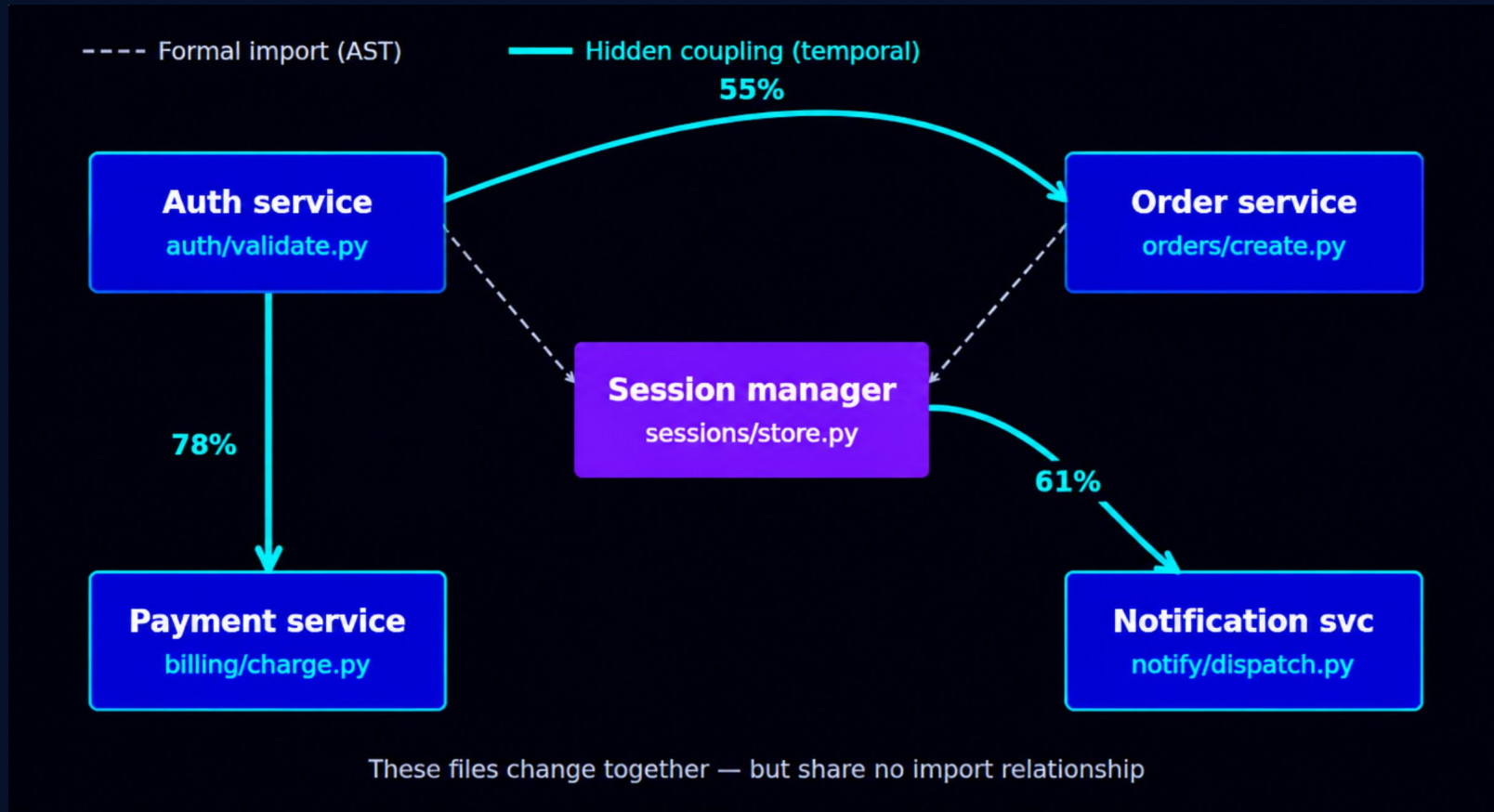
Wrapping a model with tools is not the same as engineering an agentic system.



What's In Your Security Context Graph?

- Code paths and trust boundaries
- Committers and ownership
- Git history
- Past findings and verified repairs
- Authorization flows
- Dependencies
- Routes, APIs, external calls
- Roles, access control, ...

Git Temporal Coupling



Exploit Chaining in the Real World

**Unbounded
Paid API
Abuse**

vs.

**Client-
writeable
Rate Limit**

**Server
Trust the
Counter**

**Two Paid
Upstream
APIs**

Priority: High

Priority: Med

Priority: Med

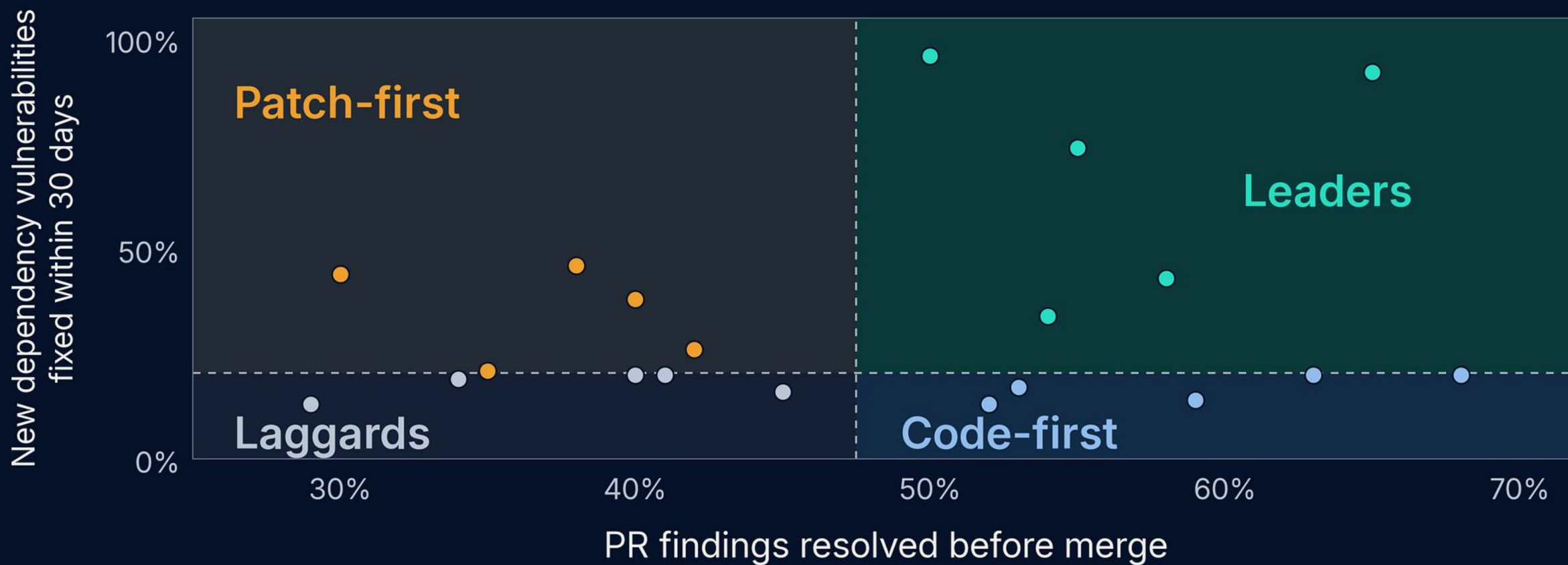
Priority: Low

Exploit Chaining

*All three discrete low/med findings go **ignored** and **unaddressed***

Example of exploit chaining for Unbounded Paid API Abuse in DryRun DeepScan and Claude Security. Test performed on September 8th, 2026

The AIDLC Security Quadrant



Suggested improvement by quadrant

Quadrant	Next improvement
Leaders	Improve priority and retain triage knowledge
Code-first	Extend repair to dependency exposure
Patch-first	Add independent checks before merge
Laggards	Name owners and protect the merge path

A paved road agents can follow

1 Challenge Merge Candidate

2 Repair

3 Re-challenge

4 Accept

Independent checks govern merge



Enterprise AI Summit
2026

James Wickett
/in/wickett · @wickett

AI Software Evolution

Phase	Need	Reliability	Coverage	Common Pitfalls
P0 - Prompts	On demand	Very low	○○○○	Inconsistent prompts
P1 - Skills	Team collab	Very low	●○○○	LLM degradation & skill bloat
P2 - Agents	Reasoning	Low	●●○○	Missing evals and harness
P3 - Factory	Verification	High	●●●●	Guardrails and coordination

AI Software Security Maturity Model

P0 Visible

Scope, owners and review coverage

P1 Codified

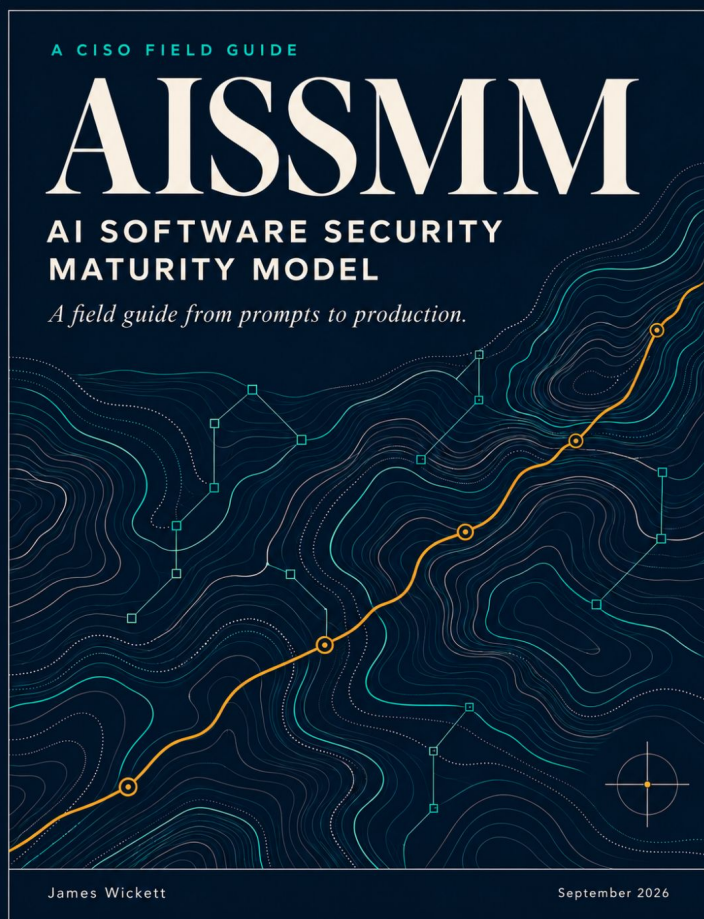
Usable policy and enforceable acceptance

P2 Operated

Reliable review, repair and recovery

P3 Assured

Maintained intelligence and reviewed decisions



AISSMM

AI Software Security Maturity Model

A field guide from prompts to production.

james+AISSMM@dryrun.security

Help I am looking for

- 1 Review and comments for the AISSMM
- 2 Dependency or library problems caused by AI coding
- 3 Case studies (named or anon) for AISSMM